

Vezérlési szerkezetek Pythonban: elágazás és ciklus

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Eddig a programjaid minden utasítást pontosan egyszer hajtottak végre, sorban. Most megtanulod, hogyan **döntsön** a program (elágazás), és hogyan **ismételjen** (ciklus). Ezzel a kettővel már igazi problémákat tudsz megoldani: a jegyzet végére prímszámokat fogsz keresni.

- **Próbáld ki, Tippelj, Kísérlet, Feladat** – ugyanúgy, mint az előző jegyzetben. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.
- **Ellenőrzés.** Minden feladatnál ott van, mit kell kapnod. Csak akkor lépj tovább, ha egyezik.
- **Hibakereső.** Ha valami nem működik, a jegyzet végén lévő táblázat segít.
- **Segítség.** A 7–9. fejezet nehezebb feladataihoz a jegyzet legvégén találsz útbaigazítást. Csak akkor lapozz oda, ha legalább öt percig próbálkoztál.

A jegyzet hosszú, és nem kell egy óra alatt a végére érned. Az 1–6. fejezet az alap; a 7–9. fejezet erre épít. A C++-os társaid ugyanezeket a feladatokat oldják meg, ugyanezzel a számozással.

1. Döntés: az if

Az **if** utasítással a program csak akkor hajt végre valamit, ha egy feltétel teljesül.

Próbáld ki

```
kor = int(input("Hány éves vagy? "))

if kor >= 18:
    print("Nagykorú vagy.")

print("Viszlat!")
```

Futtasd kétszer: egyszer 20-at, egyszer 15-öt írf be. Melyik sor jelent meg mindkétszer, és melyik csak az egyik esetben?

- **if kor >= 18:** – az **if** után áll a **feltétel**. Ez egy kérdés, amelyre a válasz igaz vagy hamis. A sor végén **kettőspont** áll.
- A következő sor **beljebb kezdődik** (négy szóközzel). Ez a behúzás jelzi, hogy a sor az **if**-hez tartozik: csak akkor fut le, ha a feltétel igaz.
- Az a sor, amely újra a sor elején kezdődik, már nem tartozik az **if**-hez: mindenképp lefut.

Az előző jegyzetben azt tanultad, hogy a sor elejére ne tegyél szóközt. Most kiderült, miért: Pythonban a behúzásnak jelentése van.

A feltételekben ezeket az összehasonlító jeleket használhatod:

Jel	Jelentése	Példa
==	egyenlő	kor == 18
!=	nem egyenlő	kor != 18
< és >	kisebb, nagyobb	kor < 18
<= és >=	legfeljebb, legalább	kor >= 18

Tippelj

Legyen `x` értéke 7. Melyik sorok jelennek meg?

```
x = 7

if x > 5:
    print("A")
if x == 5:
    print("B")
if x != 5:
    print("C")
if x <= 7:
    print("D")
```

Kísérlet - mit jelent a behúzás?

Az első programban húzd be négy szóközzel a `print("Viszlat!")` sort is, és futtasd 15-tel. Mi változott? Utána töröld ki a behúzást a `print("Nagykoru vagy.")` sor elől, és olvasd el a hibaüzenetet.

Kísérlet - a leggyakoribb hiba

Írd át az első program feltételét erre: `if kor = 18:` - egyetlen egyenlőségjellel. Mit mond a Python?

Figyelem! Az összehasonlítás jele a **dupla** egyenlőségjel: `==`. Az egyszeres `=` értékadás. Az `if` sora végéről pedig soha ne maradjon le a kettőspont!

Feladatok

1.1. feladat ★☆☆ - Pozitív

Kérj be egy egész számot. Ha pozitív, írd ki: `Pozitív szám.`

Ellenőrzés: 5 → megjelenik a szöveg; -3 és 0 → nem jelenik meg.

1.2. feladat ★☆☆ - Titkos kód

A széf kódja 1234. Kérj be egy számot, és ha egyezik a kóddal, írd ki: `Kinyilt a szef.`

Ellenőrzés: 1234 → megjelenik; 1235 → nem jelenik meg.

1.3. feladat ★☆☆ - Páros

Kérj be egy egész számot. Ha páros, írd ki: `Paros`. **Tipp:** egy szám akkor páros, ha 2-vel osztva 0 a maradéka.

Ellenőrzés: 10 → megjelenik; 7 → nem jelenik meg.

1.4. feladat ★☆☆ - Kedvezmény

Kérd be egy vásárlás összegét. Ha eléri a 10 000 Ft-ot, a vásárló 10% kedvezményt kap. Írd ki a fizetendő összeget.

Ellenőrzés: 12000 → 10800.0; 8000 → 8000; 10000 → 9000.0.

2. Kétfelé és többfelé: else, elif

Az **else** ág akkor fut le, ha a feltétel hamis. Az **elif** (az „else if” rövidítése) segítségével több lehetőség közül választhatsz.

Próbáld ki - kétfelé

```
if kor >= 18:
    print("Nagykoru vagy.")
else:
    print("Kiskoru vagy.")
```

A két ág közül mindig pontosan az egyik fut le. Az **else** ugyanott kezdődik, ahol az **if**, és utána is kettőspont áll.

Próbáld ki - többfelé

```
pont = int(input("Hany pontos lett a dolgozat? "))

if pont >= 90:
    print("jeles")
elif pont >= 75:
    print("jo")
elif pont >= 60:
    print("kozepes")
elif pont >= 40:
    print("elegseges")
else:
    print("elegtelen")
```

A program felülről lefelé vizsgálja a feltételeket, és az **első igaz** ágot hajtja végre. A többit már meg sem nézi. Próbáld ki 95, 80, 60 és 10 ponttal.

Tippelj - számít a sorrend?

Mit ír ki ez a változat 95 pontra? És miért?

```
if pont >= 40:
    print("elegseges")
elif pont >= 60:
    print("kozepes")
elif pont >= 90:
    print("jeles")
```

Feladatok

2.1. feladat ★☆☆ - Páros vagy páratlan

Kérj be egy egész számot, és írd ki, hogy páros vagy páratlan.

Ellenőrzés: 10 → **paros**; 7 → **paratlan**.

2.2. feladat ★☆☆ - A nagyobb

Kérj be két számot, és írd ki a nagyobbat. Ha egyenlők, ezt írd ki: **Egyenlok**.

Ellenőrzés: 3 és 8 → 8; 8 és 3 → 8; 5 és 5 → **Egyenlok**.

2.3. feladat ★☆☆ - Előjel

Kérj be egy számot, és írd ki, hogy pozitív, negatív vagy nulla.

Ellenőrzés: 5 → **pozitiv**; -3 → **negativ**; 0 → **nulla**.

2.4. feladat ★★☆☆ - Érdemjegy

Gépeld be a dolgozatos programot, és alakítsd át úgy, hogy a szöveg helyett a jegyet írja ki számmal.

Ellenőrzés: 95 → 5; 75 → 4; 74 → 3; 40 → 2; 12 → 1.

2.5. feladat ★★☆☆ - Belépőjegy

Egy múzeumban 6 éves kor alatt ingyenes a belépés, 6 és 17 év között 1500 Ft, 18 és 64 év között 3000 Ft, 65 évtől 2000 Ft. Kérd be a látogató életkorát, és írd ki a jegy árát.

Ellenőrzés: 5 → 0; 6 → 1500; 17 → 1500; 18 → 3000; 64 → 3000; 65 → 2000.

2.6. feladat ★★☆☆ - A legnagyobb a három közül

Kérj be három számot, és írd ki a legnagyobbat.

Ellenőrzés: 2, 9, 4 → 9; 9, 2, 4 → 9; 2, 4, 9 → 9; 5, 5, 1 → 5.

3. Összetett feltételek

Több feltételt is összekapcsolhatsz az **and** (és), az **or** (vagy) és a **not** (nem) szavakkal.

Próbáld ki

```
if kor >= 13 and kor <= 19:
    print("Tinedzser vagy.")

if kor < 6 or kor >= 65:
    print("Ingyen utazhatsz.")
```

- Az **and** akkor igaz, ha **mindkét** oldala igaz.
- Az **or** akkor igaz, ha **legalább az egyik** oldala igaz.
- A **not** megfordítja a feltételt: a **not kor >= 18** ugyanaz, mint a **kor < 18**.

Tippelj - igaz vagy hamis?

Legyen **a** értéke 5, **b** értéke 10. Töltsd ki a középső oszlopot, aztán ellenőrizd. Pythonban egy feltétel értékét közvetlenül is kiírathatod: `print(a > 3 and b > 3)`. Az igaz értéket a Python **True**, a hamisat **False** szóval jelzi.

Feltétel	A tippem	A Python szerint
<code>a > 3 and b > 3</code>		
<code>a > 7 and b > 7</code>		
<code>a > 7 or b > 7</code>		
<code>a == 5 or b == 5</code>		
<code>a != 5 and b == 10</code>		
<code>not a > 7</code>		

Kísérlet - ahogy matematikából írnád

Matematikából így írnád: $13 \leq \text{kor} \leq 19$. Próbáld ki Pythonban is:

```
if 13 <= kor <= 19:
    print("Tinedzser vagy.")
```

Futtasd 15-tel és 50-nel. Jól működik?

Tipp: Pythonban a „két érték között” feltételt a matematikai írásmóddal is megadhatod. A legtöbb programnyelvben (a társaid C++ nyelvén is) ez nem működik, ott két összehasonlítás és „és” kell hozzá.

Feladatok

3.1. feladat ★☆☆ - Határok között

Kérj be egy számot, és írd ki, hogy 1 és 10 közé esik-e (a határok is jók).

Ellenőrzés: 1 → igen; 10 → igen; 11 → nem; 0 → nem.

3.2. feladat ★☆☆ - Hétvége

Kérd be a nap sorszámát (1 = hétfő, ..., 7 = vasárnap). Írd ki, hogy hétköznap vagy hétvége. Ha a szám nem 1 és 7 közötti, ezt írd ki: **Nincs ilyen nap.**

Ellenőrzés: 3 → **hetkoznap**; 6 → **hetvege**; 7 → **hetvege**; 9 → **Nincs ilyen nap.**

3.3. feladat ★☆☆ - Mindkettővel osztható

Kérj be egy számot, és írd ki, hogy osztható-e 3-mal **és** 5-tel is.

Ellenőrzés: 15 → igen; 30 → igen; 9 → nem; 10 → nem.

3.4. feladat ★☆☆ - Háromszög

Kérd be három szakasz hosszát. Háromszög akkor szerkeszthető belőlük, ha bármelyik kettő összege nagyobb a harmadiknál. Írd ki, szerkeszthető-e.

Ellenőrzés: 3, 4, 5 → igen; 1, 2, 3 → nem; 2, 2, 5 → nem; 5, 5, 5 → igen.

3.5. feladat ★★★ - Szökőév

Egy év szökőév, ha osztható 4-gyel; de ha 100-zal is osztható, akkor csak abban az esetben, ha 400-zal is. Kérj be egy évszámot, és írd ki, szökőév-e.

Ellenőrzés: 2024 → igen; 2023 → nem; 1900 → nem; 2000 → igen.

4. Ismételtes: a while ciklus

A ciklus újra és újra végrehajt egy programrészt. A **while** ciklus addig ismételi, amíg a feltétele igaz.

Próbáld ki

```
i = 1

while i <= 5:
    print(i)
    i = i + 1

print("Vege.")
```

A **while** úgy néz ki, mint az **if**, de a behúzott rész (a **ciklusmag**) nem egyszer fut le. A végén a program visszaugrik a feltételhez, és ha még mindig igaz, újra végrehajtja.

Kövesd végig papíron! Az első két sort kitöltöttem:

i értéke	Igaz-e, hogy $i \leq 5$?	Mit ír ki?
1	igaz	1
2	igaz	2
3		
4		
5		
6		

Kísérlet - a ciklus, amely sosem áll meg

Töröld ki az `i = i + 1` sort, és futtasd a programot. Mi történik? (A programot a Ctrl+C billentyűvel vagy a futtatóablak bezárásával állíthatod meg.)

Figyelem! A ciklusmagban mindig változnia kell valaminek, amitől a feltétel egyszer hamis lesz. Különben a program soha nem áll meg: ezt hívjuk végtelen ciklusnak.

Tippelj

```
i = 10
while i > 0:
    print(i, end=" ")
    i = i - 3
```

A növelést és a csökkentést rövidebben is írhatod: az `i += 1` ugyanaz, mint az `i = i + 1`; az `i -= 3` pedig ugyanaz, mint az `i = i - 3`.

Próbáld ki - addig kérdez, amíg jó választ nem kap

```
szam = int(input("Írj be egy pozitív számot: "))

while szam <= 0:
    szam = int(input("Ez nem pozitív. Próbáld újra: "))

print("Köszönöm:", szam)
```

Itt nem tudjuk előre, hányszor fut le a ciklus: lehet, hogy egyszer sem, lehet, hogy tízszer.

Feladatok

4.1. feladat ★☆☆ - Tízíg

Írd ki a számokat 1-től 10-ig egy sorba, szóközzel elválasztva.

Ellenőrzés: 1 2 3 4 5 6 7 8 9 10

4.2. feladat ★☆☆ - Visszaszámlálás

Számolj vissza 10-től 1-ig, a végén írd ki: **Start!**

Ellenőrzés: 10 9 8 7 6 5 4 3 2 1 Start!

4.3. feladat ★☆☆ - Páros számok

Írd ki a páros számokat 2-től 20-ig.

Ellenőrzés: 2 4 6 8 10 12 14 16 18 20

4.4. feladat ★★☆☆ - Összeg nulláig

Kérj be számokat egészen addig, amíg a felhasználó 0-t nem ír be. Írd ki a beírt számok összegét és azt, hány szám volt (a nullát ne számold bele).

Ellenőrzés: 5, 8, 2, 0 → az összeg 15, 3 szám volt.

4.5. feladat ★★☆☆ - Kódtörés

A széf kódja 1234. Addig kérj be számokat, amíg a felhasználó el nem találja. Írd ki, hányadik próbálkozásra sikerült.

Ellenőrzés: 1, 99, 1234 → 3 *probalkozasra sikerult*.

4.6. feladat ★★☆☆ - Duplázás

Indulj az 1-ből, és duplázd addig, amíg meg nem haladja az 1000-et. Hány duplázás kellett, és mennyi lett a szám?

Ellenőrzés: 10 duplázás, a szám 1024.

4.7. feladat ★★☆☆ - Számjegyek

Kérj be egy pozitív egész számot. Írd ki, hány számjegyből áll, és mennyi a számjegyeinek összege.

Tipp: az utolsó számjegyet az `n % 10` adja meg; az `n // 10` pedig levágja az utolsó számjegyet.

Ellenőrzés: 2026 → 4 számjegy, az összegük 10; 7 → 1 számjegy, az összeg 7; 99999 → 5 számjegy, az összeg 45.

5. A for ciklus

Ha előre tudod, hányszor kell ismételni, a `for` ciklus kényelmesebb: a számlálót a Python kezeli helyetted.

Próbáld ki

```
for i in range(1, 6):  
    print(i)
```

Ez pontosan ugyanazt csinálja, mint az előző fejezet első programja: kiírja a számokat 1-től 5-ig. A `range(1, 6)` adja meg, milyen értékeket vegyen fel sorban az `i` változó.

Vigyázz: a `range(1, 6)` az 1-től indul, de a 6 **előtt megáll!** A végérték soha nincs benne a sorozatban.

A `range` háromféleképpen használható:

- `range(5)` – 0-tól 4-ig (öt szám, 0-tól indulva).
- `range(1, 6)` – 1-től 5-ig.
- `range(1, 10, 2)` – 1-től 9-ig, kettesével. A harmadik szám a lépésköz.

Tippelj - range-totó

Mit írnak ki ezek a ciklusok, ha a ciklusmagban mindig az `i` értékét írjuk ki? Töltsd ki, aztán ellenőrizd.

A ciklus feje	A tippem	A Python szerint
<code>for i in range(5):</code>		
<code>for i in range(2, 11, 2):</code>		
<code>for i in range(10, 0, -3):</code>		
<code>for i in range(0, 20, 5):</code>		
<code>for i in range(5, 5):</code>		

Próbáld ki - gyűjtögetés a ciklusban

A ciklus közben gyűjthetsz is. **Összegzés** – a számok összege 1-től 100-ig:

```
osszeg = 0
for i in range(1, 101):
    osszeg = osszeg + i
print(osszeg)
```

Az **osszeg** olyan, mint egy persely: üresen (0-ról) indul, és minden körben beledobjuk a következő számot.

Megszámlálás – a 7-tel osztható számok darabszáma 1 és 100 között:

```
db = 0
for i in range(1, 101):
    if i % 7 == 0:
        db += 1
print(db)
```

A legnagyobb – öt beolvasott szám közül a legnagyobb:

```
legnagyobb = int(input())

for i in range(4):
    x = int(input())
    if x > legnagyobb:
        legnagyobb = x

print("A legnagyobb:", legnagyobb)
```

Az első számot kinevezzük „eddigig legnagyobb”, a többi négyet pedig sorban összehasonlítjuk vele.

Kísérlet - hol legyen a persely?

Tedd át az összegző programban az **osszeg = 0** sort a ciklus belsejébe (behúzva, a **for** sora alá). Mit ír ki most a program? Miért éppen azt?

Feladatok

5.1. feladat ★☆☆ - Négyzetszámok

Írd ki 1-től 10-ig a számokat és a négyzetüket, soronként egyet.

Ellenőrzés: az első sor **1 1**, a harmadik **3 9**, az utolsó **10 100**.

5.2. feladat ★☆☆ - Szorzótábla

Kérj be egy számot, és írd ki a szorzótábláját 1-től 10-ig.

Ellenőrzés: 7 esetén az első sor **1 x 7 = 7**, az utolsó **10 x 7 = 70**.

5.3. feladat ★☆☆ - Összeg

Kérj be egy **n** számot, és írd ki a számok összegét 1-től **n**-ig.

Ellenőrzés: 10 → 55; 100 → 5050; 1 → 1.

5.4. feladat ★☆☆ - Faktoriális

Kérj be egy **n** számot, és írd ki a számok szorzatát 1-től **n**-ig. **Tipp:** milyen értékről induljon a gyűjtő változó, ha szorzol?

Ellenőrzés: 5 → 120; 10 → 3628800; 1 → 1.

5.5. feladat ★☆☆ - Hárommal oszthatók

Hány 3-mal osztható szám van 1 és 100 között, és mennyi az összegük?

Ellenőrzés: 33 darab, az összegük 1683.

5.6. feladat ★☆☆ - Átlag

Kérdezd meg, hány számot ad meg a felhasználó, olvasd be őket, és írd ki az átlagukat.

Ellenőrzés: 4 szám (5, 3, 4, 5) → 4.25.

5.7. feladat ★☆☆ - Legkisebb és legnagyobb

Kérj be öt számot, és írd ki a legkisebbet és a legnagyobbat.

Ellenőrzés: 4, -2, 9, 0, 7 → a legkisebb -2, a legnagyobb 9.

5.8. feladat ★★★ - Hatvány

Kérj be egy alapot és egy kitevőt, és számold ki a hatványt ciklussal (a `**` művelet most nem használható).

Ellenőrzés: 2 és 10 → 1024; 3 és 4 → 81; 5 és 0 → 1.

6. Ciklus és elágazás együtt: osztók

Ha a ciklus belsejébe elágazást teszel, a számok közül válogatni tudsz. Ebből lesz később a prímkeresés.

Próbáld ki

Egy szám osztói azok a számok, amelyekkel maradék nélkül osztható. Ez a program kiírja a beolvasott szám összes osztóját:

```
n = int(input("Melyik szám osztóit keresed? "))

for i in range(1, n + 1):
    if n % i == 0:
        print(i, end=" ")
print()
```

A ciklus végigpróbálja a számokat 1-től `n`-ig, és amelyekkel az `n` maradék nélkül osztható, azt kiírja. Figyeld meg a `range(1, n + 1)` végét: azért kell a `+ 1`, hogy maga az `n` is sorra kerüljön. Próbáld ki 12-vel, 7-tel és 36-tal.

Feladatok

6.1. feladat ★☆☆ - Osztók száma

Alakítsd át a programot: ne kiírja, hanem megszámlolja az osztókat.

Ellenőrzés: 12 → 6 osztó; 7 → 2 osztó; 36 → 9 osztó; 1 → 1 osztó.

6.2. feladat ★☆☆ - Osztók összege

Számold ki a beolvasott szám osztóinak összegét.

Ellenőrzés: 12 → 28; 7 → 8; 36 → 91.

6.3. feladat ★☆☆ - FizzBuzz

Írd ki a számokat 1-től 30-ig, soronként egyet. De a 3-mal osztható számok helyett **Fizz**, az 5-tel oszthatók helyett **Buzz**, a mindkettővel oszthatók helyett **FizzBuzz** álljon.

Ellenőrzés: a sorok eleje: 1, 2, Fizz, 4, Buzz, Fizz, 7. A 15. sor FizzBuzz. Összesen 8 sorban áll Fizz, 4 sorban Buzz, 2 sorban FizzBuzz.

6.4. feladat ★☆☆ - Közös osztók

Kérj be két számot, és írd ki a közös osztóikat. Írd ki a legnagyobbat is külön.

Ellenőrzés: 12 és 18 → a közös osztók 1 2 3 6, a legnagyobb 6.

7. Egymásba ágyazott ciklusok

Egy ciklus belsejébe másik ciklust is írhatasz. Ilyenkor a belső ciklus a külső **minden egyes** körében elejétől a végéig lefut.

Próbáld ki

```
for sor in range(1, 4):
    for oszlop in range(1, 6):
        print("*", end=" ")
    print()
```

A külső ciklus a sorokat számolja, a belső egy soron belül a csillagokat. Az üres `print()` a belső ciklus után, de a külsőn belül áll: ez zárja le a sort. Hány csillagot ír ki a program összesen?

Tippelj - milyen sorrendben?

Mit ír ki ez a program? Írd le a füzetedbe mind a hat sort, mielőtt futtatod.

```
for a in range(1, 3):
    for b in range(1, 4):
        print(a, b)
```

Figyeld meg: a külső változó (**a**) lassan változik, a belső (**b**) gyorsan – mint az óra kismutatója és nagymutatója.

Próbáld ki - a belső ciklus hossza változik

Írd át az első program belső ciklusát erre: `for oszlop in range(1, sor + 1):`. Mit rajzol most a program? Miért?

Feladatok

7.1. feladat ★☆☆ - Téglalap

Kérd be egy téglalap szélességét és magasságát, és rajzold ki csillagokból.

Ellenőrzés: 6 és 2 esetén két sor, mindegyikben hat csillag.

7.2. feladat ★☆☆ - Háromszög

Kérj be egy **n** számot, és rajzolj **n** soros háromszöget.

Ellenőrzés: 4 esetén:

```
*
**
***
****
```

7.3. feladat ★★☆☆ - Fordított háromszög

Rajzold meg a háromszöget fejjel lefelé.

Ellenőrzés: 4 esetén:

```
****
***
**
*
```

7.4. feladat ★★☆☆ - Számháromszög

Csillagok helyett számokat írf ki.

Ellenőrzés: 4 esetén:

```
1
1 2
1 2 3
1 2 3 4
```

7.5. feladat ★★☆☆ - Szorzótábla

Írd ki az 5×5 -ös szorzótáblát táblázatként. **Tipp:** az `end="\t"` (tabulátor) beállítással szép oszlopokat kapsz.

Ellenőrzés: öt sor; az első 1 2 3 4 5, az utolsó 5 10 15 20 25.

7.6. feladat ★★☆☆ - Dobókockák

Két dobókockával dobunk. Írd ki az összes olyan dobást, amelyben a két szám összege 7. Hány ilyen van? És hány olyan, ahol az összeg 10?

Ellenőrzés: 7 az összeg 6 esetben (1 6, 2 5, 3 4, 4 3, 5 2, 6 1); 10 az összeg 3 esetben.

7.7. feladat ★★☆☆ - Piramis

Rajzolj n soros piramist. A sorok elejére szóközök is kellenek!

Ellenőrzés: 4 esetén:

```
  *
 ***
*****
*****
```

7.8. feladat ★★☆☆ - Sakktábla

Rajzolj 8×8 -as sakktáblát: a sötét mezők helyén #, a világosak helyén . álljon, a bal felső sarok legyen #.

Ellenőrzés: az első sor #.#.#.#., a második .#.#.#.#, és így váltakoznak.

8. Prímszámok

Egy szám akkor **prím**, ha pontosan két osztója van: az 1 és önmaga. A 7 prím (osztói: 1, 7), a 9 nem az (osztói: 1, 3, 9), és az 1 sem prím (egyetlen osztója van).

Ebben a fejezetben lépésről lépésre építész fel egy prímkereső programot. Minden feladat az előzőre épül.

8.1. feladat ★☆☆ - Prím-e? Első változat

A 6.1. feladatban megszámoltad egy szám osztóit. Ennek alapján írd programot, amely egy beolvasott számról megmondja, prím-e.

Ellenőrzés: 7 → prím; 2 → prím; 9 → nem prím; 1 → nem prím; 97 → prím.

Próbáld ki - jelző változóval

Az osztók megszámolása helyett van egyszerűbb gondolat is: egy szám akkor **nem** prím, ha 2 és önmaga között van osztója. Elég tehát egy logikai változó, amely megjegyzi, találtunk-e ilyet. Egy logikai változó értéke **True** (igaz) vagy **False** (hamis) lehet.

```
n = int(input())

prim = True
if n < 2:
    prim = False
for i in range(2, n):
    if n % i == 0:
        prim = False

if prim:
    print(n, "prim.")
else:
    print(n, "nem prim.")
```

A **prim** változó igazról indul: „ártatlan, amíg be nem bizonyosodik az ellenkezője”. Ha a ciklus talál egy osztót, hamisra állítja. A ciklus **után** már csak meg kell néznünk, mi maradt benne.

8.2. feladat ★☆☆ - Prímek százig

Írd ki az összes prímszámot 2-től 100-ig. **Tipp:** a fenti vizsgálat kerüljön egy külső ciklus belsejébe, amely az **n** értékét lépteti 2-től 100-ig.

Ellenőrzés: a sor így kezdődik: 2 3 5 7 11 13, és 97-tel végződik.

8.3. feladat ★☆☆ - Hány prím van?

Számold meg, hány prímszám van 100-ig, és hány 1000-ig.

Ellenőrzés: 100-ig 25, 1000-ig 168.

8.4. feladat ★☆☆ - A következő prím

Kérj be egy számot, és írd ki a nála nagyobb legkisebb prímszámot.

Ellenőrzés: 20 → 23; 13 → 17; 100 → 101.

8.5. feladat ★★ - Ikerprímek

Ikerprímeknek nevezzük az olyan prímpárokat, amelyek különbsége 2, például a 11 és a 13. Írd ki az összes ikerprímpárt 100-ig.

Ellenőrzés: 8 pár: (3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73).

8.6. feladat ★★★ - Gyorsabban!

A prímvizsgálatnál nem kell n -ig próbálkozni: elég addig menni, amíg $i * i \leq n$. Gondold végig, miért! (Ha a 36-nak a 9 osztója, akkor a 4 is az: az osztók párban járnak.) Írd át a programodat, és számold meg mindkét változatban, hány osztást végez a 9973 vizsgálatakor.

Ellenőrzés: a 9973 prím. A régi változat 9971, az új 98 osztást végez.

8.7. feladat ★★★ - Prímtényezős felbontás

Kérj be egy számot, és írd ki a prímtényezős felbontását.

Ellenőrzés: $360 \rightarrow 2 \cdot 2 \cdot 2 \cdot 3 \cdot 3 \cdot 5$; $84 \rightarrow 2 \cdot 2 \cdot 3 \cdot 7$; $97 \rightarrow 97$; $1001 \rightarrow 7 \cdot 11 \cdot 13$.

9. Kihívások

Ezekben a feladatokban már neked kell kitalálnod, hogyan rakd össze az eddig tanultakat. Válaszd azt, amelyik megtetszik. Ha elakadsz, a jegyzet végén találsz segítséget.

9.1. feladat ★★☆☆ - Fibonacci-sorozat

A sorozat első két tagja 1 és 1, minden további tag az előző kettő összege. Írd ki az első 15 tagot.

Ellenőrzés: 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610

9.2. feladat ★★☆☆ - A $3n + 1$ probléma

Válassz egy pozitív egész számot. Ha páros, oszd el kettővel; ha páratlan, szorozd meg hárommal, és adj hozzá egyet. Ismételd, amíg 1-et nem kapsz. Írd ki a sorozatot, és azt, hány lépésből állt.

Ellenőrzés: $6 \rightarrow 6 \cdot 3 \cdot 10 \cdot 5 \cdot 16 \cdot 8 \cdot 4 \cdot 2 \cdot 1$, 8 lépés. A 27-ből indulva 111 lépés kell.

9.3. feladat ★★☆☆ - Fordított szám

Kérj be egy pozitív egész számot, és állítsd elő a „megfordítását” egy másik változóban, számolással. Írd ki azt is, hogy a szám visszafelé olvasva ugyanaz-e.

Ellenőrzés: $1234 \rightarrow 4321$, nem ugyanaz; $12321 \rightarrow 12321$, ugyanaz.

9.4. feladat ★★★ - Legnagyobb közös osztó

A 6.4. feladatban végigpróbáltad az összes számot. Az ókori görögök ennél sokkal gyorsabb módszert ismertek: amíg a nem nulla, számold ki az a és b osztási maradékát; az a új értéke legyen b , a b új értéke a maradék. A végén az a a legnagyobb közös osztó.

Ellenőrzés: 48 és 18 $\rightarrow$ 6; 100 és 75 $\rightarrow$ 25; 17 és 5 $\rightarrow$ 1.

9.5. feladat ★★★ - Tökéletes számok

Egy szám tökéletes, ha megegyezik a nála kisebb osztóinak összegével: $6 = 1 + 2 + 3$. Keresd meg az összes tökéletes számot 1000-ig.

Ellenőrzés: három ilyen szám van: 6, 28, 496.

9.6. feladat ★★★ - Armstrong-számok

A 153 különleges szám: $1 \cdot 1 \cdot 1 + 5 \cdot 5 \cdot 5 + 3 \cdot 3 \cdot 3 = 153$. Keresd meg az összes háromjegyű számot, amely megegyezik a számjegyei köbének összegével.

Ellenőrzés: négy ilyen szám van: 153, 370, 371, 407.

9.7. feladat ★★★ - Pitagoraszai számhármasok

Keresd meg az összes olyan a, b, c egész számhármas, amelyre $a < b < c$, egyik sem nagyobb 20-nál, és $a \cdot a + b \cdot b = c \cdot c$.

Ellenőrzés: hat hármas van: (3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15), (12, 16, 20).

9.8. feladat ★★★ - Goldbach-sejtés

A sejtés szerint minden 2-nél nagyobb páros szám felírható két prímszám összegeként. Kérj be egy páros számot, és írd ki azt a felbontását, amelyben az első prím a lehető legkisebb.

Ellenőrzés: 28 → 5 + 23; 100 → 3 + 97; 4 → 2 + 2.

9.9. feladat ★★★ - Az n-edik prím

Kérj be egy `n` számot, és írd ki az `n`-edik prímszámot.

Ellenőrzés: az 1. prím a 2; a 10. prím a 29; a 100. prím az 541.

Hibakereső: mi lehet a baj?

Az elágazások és ciklusok hibái gyakran nem járnak hibaüzenettel: a program lefut, csak mást csinál, mint amit szeretnél. Keresd meg a táblázatban, amit tapasztalsz.

Amit látsz	Mit jelent?	Mit tegyél?
<code>SyntaxError: expected ':'</code>	Lemaradt a kettőspont.	Az <code>if</code> , <code>elif</code> , <code>else</code> , <code>while</code> és <code>for</code> sora végére kettőspont kell.
<code>IndentationError: expected an indented block</code>	A kettőspont utáni sor nincs behúzva.	Húzd be négy szóközzel, ami az elágazáshoz vagy a ciklushoz tartozik.
<code>IndentationError: unindent does not match</code> vagy <code>unexpected indent</code>	A behúzások nem egyformák.	Mindig pontosan négy szóközt használj szintenként.
<code>SyntaxError: invalid syntax. Maybe you meant '=='</code>	Egyszeres <code>=</code> áll a feltételben.	Összehasonlításhoz <code>==</code> kell.
A program nem áll meg, vég nélkül ír vagy „lefagy”	Végtelen ciklus: a feltétel soha nem lesz hamis.	Állítsd le (Ctrl+C). Változik a ciklusváltozó? Jó irányba?
A ciklus egyszer sem fut le	A feltétel már az elején hamis, vagy a <code>range</code> üres.	Nézd meg a kezdőértéket és a <code>range</code> két végét.
Egyel kevesebb kör fut le, mint kellene	A <code>range</code> végértéke nincs benne a sorozatban.	<code>range(1, n + 1)</code> megy 1-től <code>n</code> -ig.
Egy utasítás minden körben lefut, pedig csak a végén kellene (vagy fordítva)	Rossz a behúzás.	Ami a ciklus után jön, annak a behúzása egyezzen a <code>for</code> vagy <code>while</code> szóéval.
Az összeg vagy a darabszám hibás (például az utolsó érték jelenik meg)	A gyűjtő változót a cikluson belül állítod be, így minden körben újraindul.	Az <code>osszeg = 0</code> kerüljön a ciklus elé.
<code>NameError: name 'osszeg' is not defined</code>	A gyűjtő változónak nincs kezdőértéke.	Hozd létre a ciklus előtt.
Az alakzat egyetlen hosszú sorba kerül	Hiányzik a sort lezáró <code>print()</code> .	A belső ciklus után, a külsőn belül legyen.
Minden csillag külön sorba kerül	Kimaradt az <code>end=""</code> , vagy a <code>print()</code> a belső ciklusba került.	Ellenőrizd a belső <code>print</code> beállítását és a behúzásokat.

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>if feltétel:</code>	a behúzott rész csak akkor fut le, ha a feltétel igaz
<code>if ...: és else:</code>	kétfelé ágazás
<code>if ...:, elif ...:, else:</code>	többfelé ágazás: az első igaz ág fut le
<code>==, !=, <, >, <=, >=</code>	összehasonlítás
<code>and, or, not</code>	és, vagy, nem
<code>while feltétel:</code>	ismétlés, amíg a feltétel igaz
<code>for i in range(5):</code>	számlálós ciklus 0-tól 4-ig
<code>for i in range(1, 11):</code>	számlálós ciklus 1-től 10-ig
<code>for i in range(10, 0, -1):</code>	visszafelé 10-től 1-ig
<code>i += 1 és i -= 1</code>	növelés és csökkentés eggyel
<code>összeg = 0</code> a ciklus előtt	gyűjtő változó összegzéshez
<code>van = False</code> a ciklus előtt	jelző változó eldöntéshez
<code>n % i == 0</code>	az <code>n</code> osztható <code>i</code> -vel

Figyelem! Három dolog, amit soha ne felejts el: a behúzás mutatja meg, mi tartozik az elágazáshoz és a ciklushoz; a `range` végértéke nincs benne a sorozatban; a gyűjtő és a jelző változókat a ciklus **előtt** hozd létre.

Segítség a feladatokhoz

Csak akkor olvasd el egy feladat segítségét, ha már legalább öt percig próbálkoztál vele. A segítség nem megoldás: azt mondja meg, merre indulj.

4-6. fejezet

- **4.7.** `while n > 0`: ciklus: minden körben add hozzá az összeghez az `n % 10` értékét, növel a számlálót, és oszd el az `n`-et tízzel egész osztással (`n = n // 10`).
- **5.4.** Szorzásnál a gyűjtő változó 1-ről induljon, ne 0-ról.
- **5.8.** A kitevő azt mondja meg, hányszor kell az eredményt megszorozni az alappal. Az eredmény 1-ről induljon.
- **6.3.** Először a „mindkettővel osztható” esetet vizsgálj, különben soha nem jut rá sor (emlékezz a 2. fejezet sorrendes kísérletére).
- **6.4.** A ciklus menjen 1-től a kisebbik számig; a feltétel: mindkét számot osztja. A legnagyobb közös osztó az utoljára talált közös osztó.

7. fejezet

- **7.3.** Kétféle megoldás van: a külső ciklus számolhat visszafelé (`range(n, 0, -1)`), vagy a belső ciklus hossza lehet `n - sor + 1`.
- **7.4.** A belső ciklus változóját írd ki, ne csillagot; a beállítás `end=" "` legyen.
- **7.5.** A kiírandó érték a két ciklusváltozó szorzata.
- **7.6.** Két egymásba ágyazott ciklus 1-től 6-ig (`range(1, 7)`), bennük egy `if` az összegre és egy számláló.
- **7.7.** Minden sorban két belső ciklus kell egymás után: az első szóközöket ír (`n - sor` darabot), a második csillagokat (`2 * sor - 1` darabot).
- **7.8.** Egy mező sötét, ha a sor és az oszlop sorszámának összege páros.

8. fejezet

- **8.1.** Számold meg az osztókat (6.1. feladat), és a ciklus után vizsgálj meg: ha pontosan 2 van, a szám prím.
- **8.2.** A külső ciklus: `for n in range(2, 101)`. A belsejébe, behúzva kerül a jelző változós vizsgálat. Fontos: a `prim = True` sor a külső ciklus **belsejében** legyen, hogy minden számnál újra igazról induljon.
- **8.3.** Ugyanaz, mint a 8.2., de kiírás helyett számlálót növelsz.
- **8.4.** Vizsgálj sorban a `szam + 1`, `szam + 2`, ... számokat, amíg prímet nem találsz. Ehhez `while` ciklus kell, amelynek a feltétele egy jelző változó: „még nem találtam”.
- **8.5.** Jegyezd meg az előző megtalált prímet. Ha az új prím pontosan 2-vel nagyobb nála, találtál egy párt.
- **8.6.** A `for` ciklus helyett `while` kell: `i = 2`, majd `while i * i <= n`; a ciklusmag végén `i += 1`. Az osztások megszámlálásához tegyél egy számlálót a ciklusba.
- **8.7.** Indulj a `p = 2` osztóval. Amíg az `n` nagyobb 1-nél: ha `n` osztható `p`-vel, írd ki a `p`-t, és oszd el vele az `n`-et egész osztással (a `p` marad!); ha nem osztható, növelj a `p`-t.

9. fejezet

- **9.1.** Két változóban tartsd az utolsó két tagot (**elozo** és **mostani**). Minden körben számold ki az összegüket egy harmadik változóba, aztán „léptesd” a kettőt.
- **9.2.** **while n != 1:** ciklus, benne **if - else** a páros és a páratlan esetre, és egy lépésszámláló. A felezéshez egész osztást használj (**n // 2**).
- **9.3.** A 4.7. feladat ciklusa kell. A megfordított szám építése: **forditott = forditott * 10 + n % 10**. Az eredeti számot tedd el egy másik változóba, mert a ciklus „elfogyasztja”.
- **9.4.** **while b != 0:** ciklus. Kell egy segédváltozó a maradéknak, mert az **a** és a **b** értéke egyszerre változik.
- **9.5.** Külső ciklus a számokra 2-től 1000-ig, belső ciklus az osztók összegére (6.2. feladat, de a számot magát ne add hozzá).
- **9.6.** Ciklus 100-tól 999-ig. A három számjegy: **n // 100**, **n // 10 % 10** és **n % 10**.
- **9.7.** Három egymásba ágyazott ciklus. Ha a **b** ciklusa **a + 1**-ről, a **c** ciklusa **b + 1**-ről indul, az **a < b < c** feltétel magától teljesül.
- **9.8.** Próbáld végig a **p** értékeit 2-től: az első olyan **p** a megoldás, amelyre a **p** és az **n - p** is prím. Két prímvizsgálat kell egymás után, két külön jelző változóval.
- **9.9.** Számold a megtalált prímeket egy **while** ciklusban, amíg a darabszám el nem éri az **n**-et.